Hypermedia Systems & Architecture

Rimbert Rudisch-Sommer

rimbert.rudisch-sommer@fh-hagenberg.at

Recap: JavaScript Functions

- Functions used as value:
  - anonymous function expression

```
let a = function() { ... }
```

```
let a = () => {}
```

  - named function expression
    - name available only from within the function, useful eg. for recursion

```
let a = function a() { ... }
```

- Self-Evaluating functions
  - aka. IIFE = immediately invoked function expression

```
let a = (function() {return 1;})()      ==> a = 1
```

- Inner functions are allowed

```
function foo () {
  function bar() {
  }
}
```

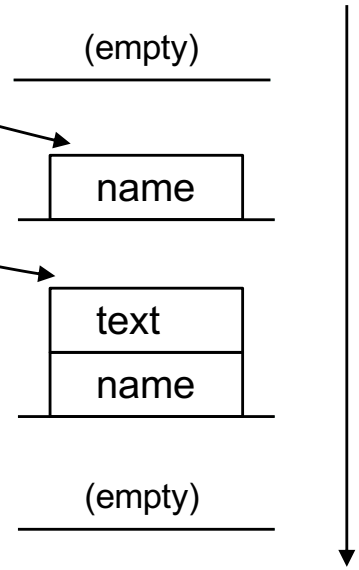
Recap: Functions & local Scope

- Lexical (static) Scoping
  - local name scope (let / var / const)
- the JavaScript engine manages an internal datastructure for storing local variables
- local variables of a function are
  - created on the stack when they are declared
  - removed from the stack when the function ends

```
function greeting(name) {

  let text = "Hello " + name + "!";

  return text;
}
```



Recap: Closures

- special programming concept concerning "inner" functions and their access of local variables
  - usually local variables of a function are removed when the function ends

- Javascript treats functions as objects
  - functions can be assigned to variables

```
let a= function() {alert('hello')};
a(); // call the function via variable
```

  - functions can be passed as parameters

```
button.addEventListener('click', function(event) {...})
```

  - functions can be used as return values

```
function test1() {
  return function() {alert('hello');} // anonymous inner function!
}
let b= test1();
b(); // call the returned function via variable
```

- A closure is created when
  - an inner function references local variables of the outer function
  - while the outer function has ended.

Recap: Closures

- if a function is used as return value, it keeps the variables of the outer function even after the outer function has ended

```
function makeClosure() {
  let text="Hello ";
  return function(name) {alert(text+name);} // anonymous inner function
}

let simpleHello= makeClosure();  // outer function ends,
                                 // variable text kept in a closure

simpleHello("World!");  // calls returned inner function
                        // Output: "Hello World!"
```

- this can be used to parameterize the returned function:

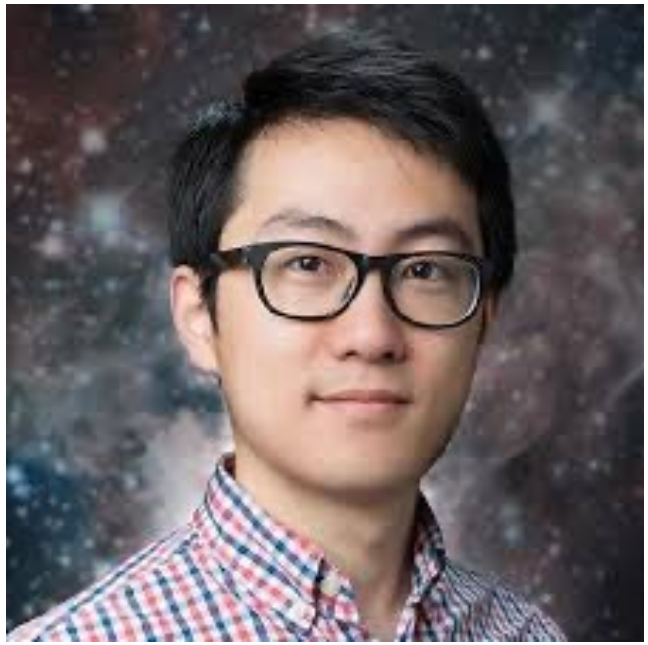
```
function createParamGreeting(greeting) {      // parameter has same scope as local var
  return function(name) {alert(greeting + name);}
}

let helloGreeting= createParamGreeting("Hello ");
helloGreeting("World!");  // alerts "Hello World!"

let howdyGreeting= createParamGreeting("Howdy ");
howdyGreeting("again"); // alerts "Howdy again!"
```

Recap: Vue.js ("view")

- Progressive JS framework for building user interfaces
  - Incrementally adoptable
- Created by Evan You, first released Feb 2014
- http://viewjs.org/
- Declarative rendering (templates)
- Easy event binding
- "Reactive" two-way binding
  - Model <-(updates)-> UI Element
- Component-based
- Incremental



Recap: What does a typical JavaScript application do?

```
[…]
let notesData = [];

function renderData(){
    let output = "";
    for(let entry of notesData){
        output += `<em>${entry.date}</em> -- ${entry.content}<br>`;
    }
    document.querySelector("#notes").innerHTML = output;
}

function saveData() { […] renderData();}
function loadData() { […] renderData();}

document.querySelector("#save").addEventListener('click', saveData);
loadData();
```

manage some data

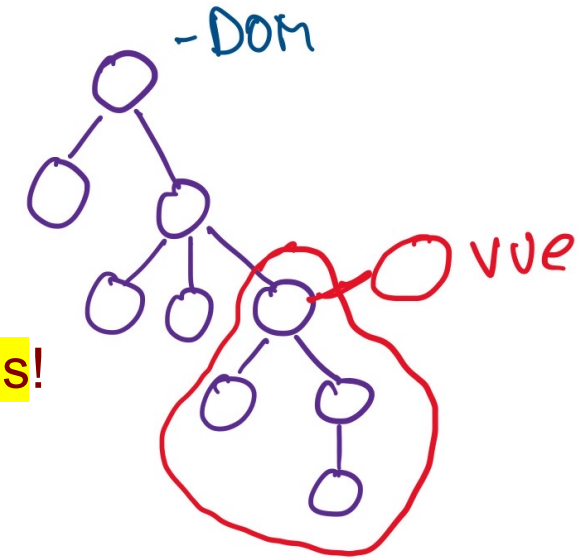
render the data on the webpage

change the data on certain events, eg. user input (and re-render)

Recap: Vue Basic Idea

- Vue lets you create instances of itself

- a Vue instance
  - is attached to a node of the DOM tree (HTML element)
  - manages data
  - renders the data in the subtree of the node
    - declaratively via templates
  - automatically re-renders when the data changes!
  - manages functions (event handlers)
  - receives events occuring in the subtree of the attached node
    - to be able to manipulate the data



Recap: Vue Instance & Declarative Rendering

- A Vue Instance is created via **`Vue.createApp()`**
  - takes options object as parameter (Options API)
  - "mounted" at a DOM element

```
Vue.createApp({
  data() {
    return { message: 'Hello HYP3!' };
  }
}).mount('#app');
```
JavaScript

data – function returning "variables" of the instance,

mounted at element '#app'

- declaratively render data to the DOM using *templates*:

```
<div id="app">
  {{ message }}
</div>
```
HTML

- every change of the instance data also updates DOM automatically (*reactivity system*)
  - Here: if **`message`** gets changed in JavaScript, the value changes on the web page also automatically

Recap: Templates, Directives

- Plain text: "mustache syntax" `{{ message }}`
  - meta characters (<,>, …) are escaped (< , > , …)
  - Single JavaScript expressions are allowed

- Directives: Special attributes with v- prefix
  - Values must be single JavaScript expressions
  - Arguments denoted by colon
- v-html - does not escape meta characters (<,>,…)
- v-bind (:) - Bind attribute values reactively

```
<div id="app">
  <span v-bind:title="message">Vue.js</span>
</div>
```

- v-on (@) - Set event handler (example see next slide)

Recap: Event Handling

- Define functions with Vue instance option *methods* and attach them as event handlers with directive *v-on*:

```
Vue.createApp({
  data() {
    return {message: 'Hello HYP3!'};
  },
  methods: {
    plusMessage() {
      this.message = this.message+' again';
    }
  }
}).mount('#app');
```

```
<div id="app">
  <p>{{ message }}</p>
  <button v-on:click="plusMessage">Enhance Message</button>
</div>
```

Shortcut: `@click="plusMessage"`

Recap: Vue Data

- *on creation* of the Vue instance, all elements of its *data property* are added to *reactivity system*
  - automatic DOM updates
  - elements added later (after initial creation) will not be reactive

- user-defined properties are accessible via dot-notation
- vue properties are accessible via prefix $

```
let app = Vue.createApp({
  data() {
    return { message: 'Hello MTD280!' };
  }
}).mount('#app');

app.message = "New Value";
app.$el // readonly, DOM Object with ID '#app'
```

Recap: Conditional Rendering

- v-if

```
<h3 v-if="ok">Yes</h3>                    // ok is boolean (true/false)
```

- v-else (must follow immediately after v-if)

```
<h3 v-if="ok">Yes</h3>
<h3 v-else>No</h3>
```

- v-else-if

```
<h3 v-if="type === 'A'">Type A</h3>
<h3 v-else-if="type === 'B'">Type B</h3>
<h3 v-else>Not Type A or B</h3>
```

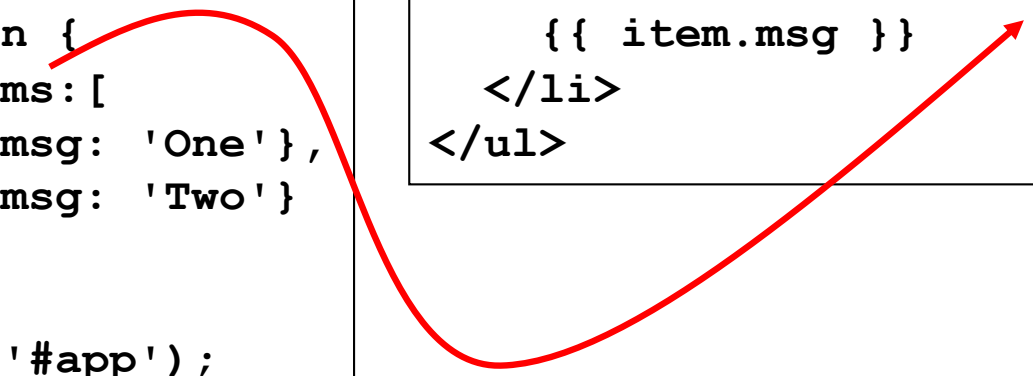
- v-show
  - like v-if, but DOM element is always rendered but possibly hidden (display:none)

Recap: List Rendering

- v-for
  - Special value syntax: *item in items*

```
Vue.createApp({
  data() {
    return {
      items:[
        {msg: 'One'},
        {msg: 'Two'}
      ]
    }}
}).mount('#app');
```

```
<ul id="app">
  <li v-for="item in items">
    {{ item.msg }}
  </li>
</ul>
```



  - Use

    ```
    v-for="(item, index) in items"
    ```

    if the indizes of the elements are needed (names of loop variables can be chosen individually)

  - Vue also wraps the following array's mutation methods for reactiveness: push(), pop(), shift(), unshift(), splice(), sort(), reverse()

Recap: Form Input

- v-model
  - "two-way binding"



```
Vue.createApp({
  data() {
   return { message: '' };
  }
}).mount('#app');
```

```
<div id="app">
  <input type="text" v-model="message">
  <p>Message is: {{ message }}</p>
</div>
```

- Also for textarea, checkbox, radiobuttons, select, …

Recap: Lifecycle Hooks

- Methods with a given name
  - called automatically at certain points in the lifetime of a Vue instance
- beforeCreate(), created()
  - before/after setting up reactivity system
  - only called once
- beforeMount(), mounted()
  - before/after setting up and syncing virtual dom (el gets connected)
  - maybe called multiple times
- beforeUpdate(), updated()
  - before/after re-rendering updated values
- beforeUnmount(), unmounted()
  - before/after tearing down subelements/reactivity system

Recap: Computed Properties

- Vue option "computed"
- to compute a value from existing data
  - the function is used as getter function
  - value gets cached, only changes when dependencies change

```
Vue.createApp({
  data() {
    return {
      firstName: '…',
      lastName: '…'
  }},
  computed: {
    fullName() {
      return this.firstName + " " + this.lastName;
  }}}).mount('#app');
```

```
<div id="app">
  <p>{{ fullName }}</p>
</div>
```

Recap: Class / Style Bindings

- v-bind:class, v-bind:style
  - Object Syntax: toggle styles

```
data(){ return {
  isActive: true,
}}
```

```
<div class="static"
   v-bind:class="{ active: isActive }" >
</div>
```

  - Array Syntax: apply styles

```
data(){ return {
  myClass: 'flashy',
}}
```

```
<div class="static"
   v-bind:class="[myClass]">
</div>
```

  - mixing object and array style

```
<div v-bind:class="[{ active: isActive }, myClass]" >
</div>
```

Recap: Virtual DOM

- it's "expensive" to update the DOM
  - node lookup (eg by id) in thousands of nodes
  - re-render after change

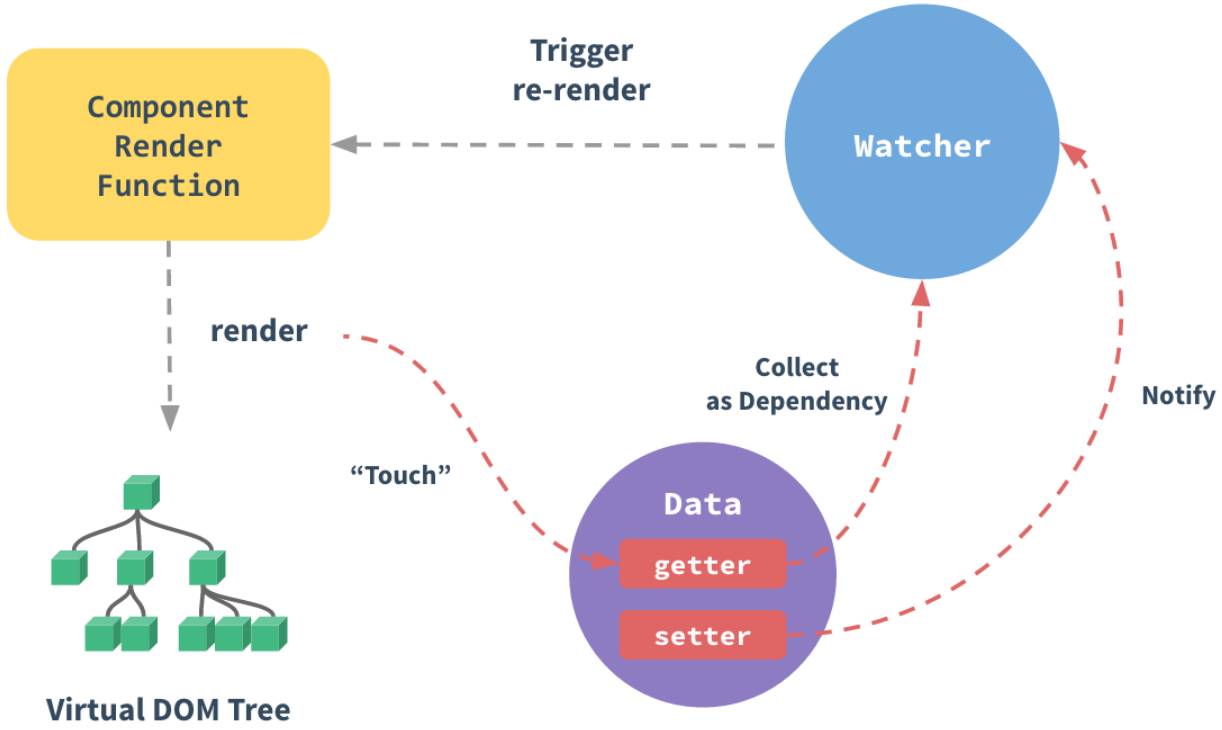
- a lot of small, frequent changes will slow the page down

- create a data structure similar to dom (aka. virtual dom)
  - representing the parts of the page we want to change
  - changing the virtual dom is less expensive
  - only sync with real dom at certain times -> less re-renderung

```
let domNode = {
  tag: 'ul',
  attributes: { id: 'myId' },
  children: [ // where the LI's would go ]
};
```

Recap: Reactivity

- On initialization Vue iterates over all data properties
- Converts them to getter/setter methods / creates proxy objects (Vue3)
- Invisible to the user dependency tracking is enabled
- Every instance has a "watcher" recording properties used in rendering
- When one of the setter methods is called triggers re-render



Recap: $refs, $nextTick()

- $refs
  - to access html elements from Vue instance programmatically

```
<input ref="myInput" ...>

$refs.myInput.select();
$refs['myInput'].select();
```

- $nextTick(callback)
  - defer the callback to be executed after the next DOM update
    - use it after you've changed some data to wait for the DOM update

```
<div ref="msg">{{ message }}</div>          // data: {message: 'old'}

this.message = 'new';
console.log(this.$refs.msg.textContent);    // 'old'
this.$nextTick(function() {
  console.log(this.$refs.msg.textContent)}); // 'new'
```

The Canvas Element

- "The canvas element provides scripts with a *resolution-dependent bitmap canvas*, which can be used for rendering graphs, game graphics, or other visual images on the fly."

- canvas Attributes
  - width, height and general html-Attributes (id, name, …)
- Example:

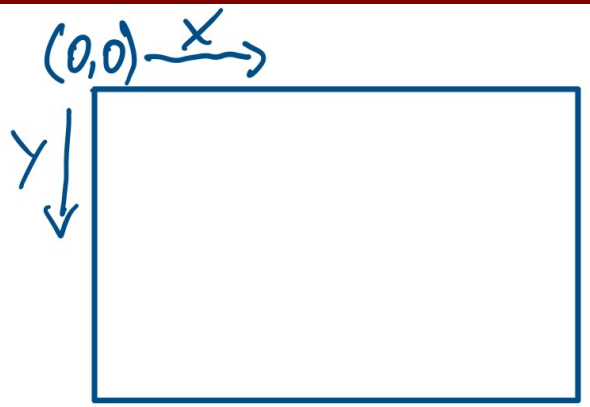
```
<canvas id="c" width="200" height="200">
</canvas>
```

- DOM-Function: getContext()
  - Context: Drawing API ('2d' / 'webgl')

```
var ctx= document.getElementById('c').getContext('2d');
```

Canvas 2D Context

- Coordinate system:
  - (0,0) upper left corner
  - y down, x to the right



- Overview API Methods
  - Rectangles
  - Styles (fill, stroke; supporting gradients)
  - Text
  - Paths
  - Images
    - drawImage
  - Transformation
    - rotation, scaling, translation, combined: transformation
  - Pixel API
    - getImageData, putImageData

Rectangles

- ctx.strokeRect(x,y,w,h) - draw border only
- ctx.fillRect(x,y,w,h) -
- ctx.clearRect(x,y,w,h)

```
<!DOCTYPE html><html lang="de">
<head><meta charset="utf-8" /><title>Canvas: Rectangles</title>
<script type="text/javascript">

function draw() {
 ctx= document.getElementById('c').getContext('2d');
 ctx.fillRect(10,10,100,50);
 ctx.strokeRect(10,100,100,50);
 ctx.clearRect(50,20,10,10);
}

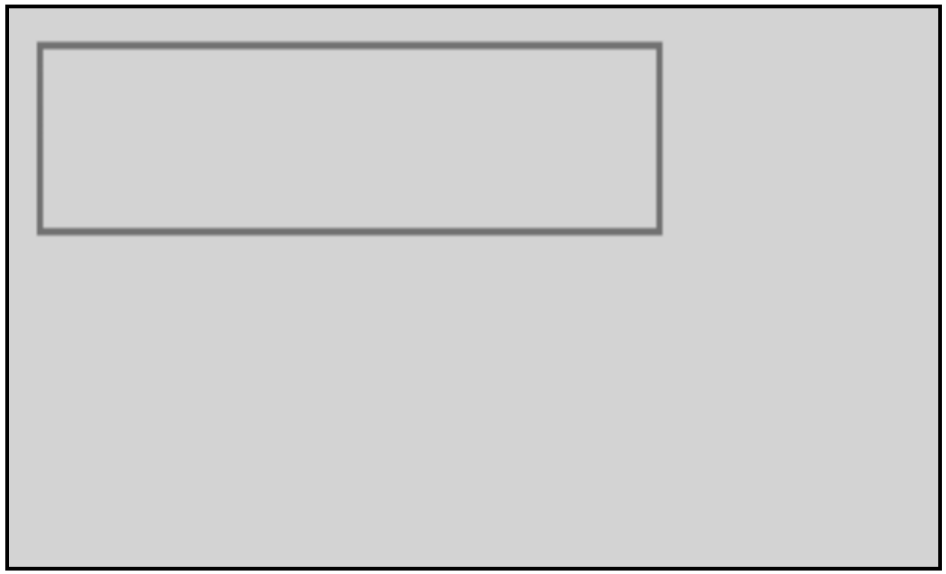
</script></head>
<body onload="draw()">
<h1>Canvas: Rectangles</h1>
<canvas id="c" width="400" height="400"></canvas>
</body></html>
```

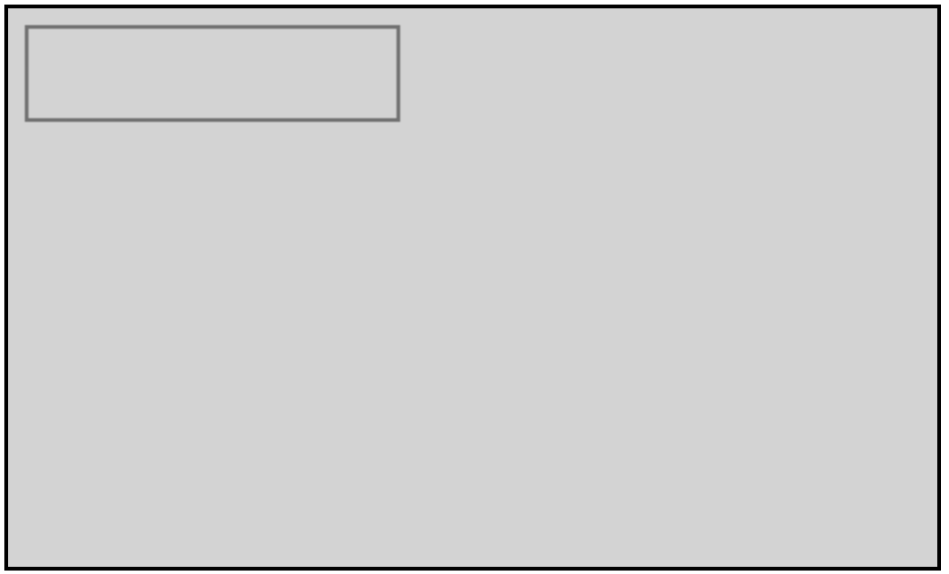
Dimensions: Attribute vs. Style

- Attribut width/height: defines dimensions of canvas coordinate system
  - default: 150h 300w

- Style width/height: changes display size
  - default: same as attribute (s. above)
  - if different to attribute values: distortion

w/h: Attribute vs. Style





```
<canvas id="c1" style=
"height:300px; width=500px">
</canvas>
```

```
<canvas id="c2"
height="300" width="500">
</canvas>
```

```
let ctx =
  document.querySelector("#c1")
  .getContext("2d");

 ctx.rect(10,10,200,50);
 ctx.stroke();
```

```
let ctx =
  document.querySelector("#c2")
  .getContext("2d");

 ctx.rect(10,10,200,50);
 ctx.stroke();
```

Line / Fill / Stroke Styles

- ctx.lineWidth
  - pixel
- ctx.fillStyle, ctx.strokeStyle
  - possible values:
  - color (vgl. CSS: red, #FF0000, rgb(255,0,0), …)
  - CanvasGradient (linear, radial)
  - CanvasPattern

- gradient= ctx.createLinearGradient(x0,y0,x1,y1)
- gradient= ctx.createRadialGradient(x0,y0,r0,x1,y1,r1)
- gradient.addColorStop(offset, color)
  - 0.0 <= offset <= 1.0

- ctx.createPattern(image, repetition)
  - image: image-,canvas-,(video-)Element
  - repetition: repeat, repeat-x, repeat-y, no-repeat

Text

- ctx.font
  - like CSS font-property
- ctx.textAlign, ctx.textBaseline

- ctx.fillText(text,x,y[,maxWidth])
- ctx.strokeText(text,x,y[,maxWidth])

- metrics= ctx.measureText(text)
  - uses current font properties
- metrics.width

Paths

- Definition of a series of lines
  - ctx.beginPath() context-default-path
  - p= new Path() custom path- Object
  - closePath()
- draw a path:
  - ctx.stroke(), ctx.stroke(p)
  - ctx.fill(), ctx.fill(p)
  - ctx.isPointInPath([p],x,y)
- path definition
  - used on default- or custom path object
  - moveTo(x,y), lineTo(x,y)
  - quadraticCurveTo(cpx,cpy,x,y)
  - bezierCurveTo(cp1x,cp1y,cp2x,cp2y,x,y)
  - arcTo(x1,y1,x2,y2,radiusX[,radiusY,rotation])
  - arc(x,y,radius,startAngle,endAngle[,anticlockwise])
  - ellipse(x,y,radiusX,radiusY,rotation,startAngle,endAngle,anticlockwise)
  - rect(x,y,w,h)

Images

- Embedding of Image-, Canvas-, Video-Elements
- ctx.drawImage(image,dx,dy)
- ctx.drawImage(image,dx,dy,dw,dh)
- ctx.drawImage(image,sx,sy,sw,sh,dx,dy,dw,dh)

Pixel

- accessing the single pixels via a data structure
- d= ctx.getImageData(sx,sy,sw,sh)
- d.width, d.height
- d.data
  - one dimensional array of pixels as RGBA-values [r1,g1,b1,a1,r2,g2,b2,a2,…]

- d= ctx.createImageData(sw,sh)
- d= ctx.createImageData(d2)

- ctx.putImageData(d,dx,dy)

Transformations

- ctx.scale(x,y)
- ctx.rotate(angle)
- ctx.translate(x,y)

- ctx.transform(a,b,c,d,e,f)
  - applies transformation matrix
  - a,d: scale-x,scale-y
  - b,c: skew-x,skew-y
  - e,f: translate-x, translate-y

- ctx.setTransform(a,b,c,d,e,f)
  - first resets to identity matrix

Animation

- repeating calls of a draw-Method()
- was: setInterval(f,ms)
  - calls function f every ms milliseconds
  - even, if window is hidden
  - animation redraw independent from other browser rendering -> inefficient
- today: requestAnimationFrame()
  - browser calls draw function in optimal way

Animation Loop

```
function draw() {

  // draw code:
  ...

  // continue animation:
  requestAnimationFrame(draw);
}

// start animation:
requestAnimationFrame(draw);
```

Canvas: toDataURL

- function of the canvas element (not the context!):
  - for storing the canvas content
- URL spec allows embedding of base64 coded assets
  - to reduce number of requests
  - URL-Syntax: "data:"<mimetype>";"<encoding>","<data>
  - Example: <….encoded data …>

```
function draw() {
 let canvas= document.getElementById('c');
 let ctx= canvas.getContext('2d');
 ctx.fillRect(10,10,100,50);
 ctx.strokeRect(10,100,100,50);
 ctx.clearRect(50,20,10,10);
 // store image to url and show url:
 window.location= canvas.toDataURL('image/png');
}
```

Alternative 2D Graphics: SVG

- Scalable Vector Graphics
  - describe graphics in XML
  - resolution independent
  - each shape is an object – scriptable / styleable

```
<svg width="400" height="400">
    <rect width="200" height="50" style="fill:#FF0000;stroke-width:2;stroke:#000000" />
</svg>
```

- Shapes: line, rectangle, circle, ellipse, polyline, polygon
- Paths: moveto, lineto, closepath, curves (Bezier, Elliptical Arc), …
- Further: Texts, Groups, Embedded Images, Filters, …

Recap: Vue Data

- on creation of the Vue instance, all properties of its data object are added to *reactivity system*
- user-defined properties accessible via dot-notation, eg. this.message
- instance properties accessible via prefix $, eg. this.$el
- properties added after initial creation will not be part of the reactivity system
  - eg. in created() or mounted() lifecycle functions
  - cp. canvas example:

```
…
<div id="app">
  <canvas ref="cvs" />
</div>
```

```
Vue.createApp({
  data() {return {}},
  methods: {
    // can access this.ctx …
  },
  mounted() {
    this.ctx = this.$refs.cvs.getContext('2d');
  }
}).mount("#app");
```

Asynchronity

- Where is the error in this canvas example:

```
…
<div id="app">
  <canvas ref="cvs" />
</div>
...
```

```
Vue.createApp({
  data() {return {}},
  methods: {
    drawBoard() {
      let img;
      img = document.createElement("img");
      img.src = "background.jpg";
      this.ctx.drawImage(img,0,0);
      …
    }
  },
  mounted() {
    this.ctx =
      this.$refs.cvs.getContext('2d');
    this.drawBoard();
  }
}).mount("#app");
```

! JavaScript Event Loop !

- JS is single threaded, not interruptable
- does not block on I/O operations but uses events
  - starts the I/O operation and continues with next line of code
  - if I/O operation is completed the system creates an event
- JavaScript Engine lets you register event handler functions
- JavaScript Engine stores all incoming events in a queue
  - first in first out (FIFO)

- when "idle": takes the first event available, executes the handler (w/o interruption)
- takes the next event available, executes the handler...

- a handler is never interrupted by a new event from the outside!

View Asynchronity Example

- Where is the error in this canvas example:

```
…
<div id="app">
  <canvas ref="cvs" />
</div>
...
```

```
Vue.createApp({
  data() {return {}},
  methods: {
    drawBoard() {
      let img;
      img = document.createElement("img");
      img.src = "background.jpg";
      this.ctx.drawImage(img,0,0);
      …
    }
  },
  mounted() {
    this.ctx =
      this.$refs.cvs.getContext('2d');
    this.drawBoard();
  }
}).mount("#app");
```

Solution:

- "Problem" is the line
  `img.src = "background.jpg";`
- Loading the image is an I/O operation
- JavaScript executes them asynchronosly, ie.
  - Does not wait for the image to load BUT
  - Will send an event (onload), when it has finished loading
- One correct solution:

```
drawBoard() {
  let img;
  let self = this;
  img = document.createElement("img");
  img.onload = function() {
      self.ctx.drawImage(img,0,0);
  };
  img.src = "background.jpg";
  …
}
```

  - this/self: as the event handler will be called by the system, "this" is set to the calling object (global window object); notice the closure keeping the value of "self"

Solution:

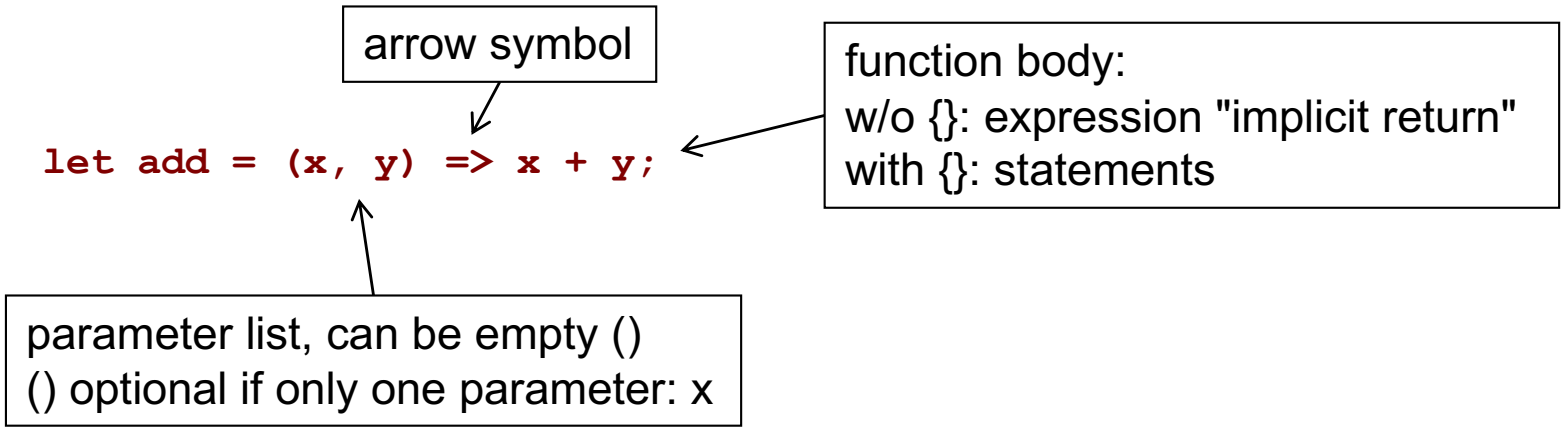
```
…
<div id="app">
  <canvas ref="cvs" />
</div>
...
```

```
Vue.createApp({
  data() {return {}},
  methods: {
    drawBoard() {
      let img;
      let self = this;
      img = document.createElement("img");
      img.onload = function() {
        self.ctx.drawImage(img,0,0);
      };
      img.src = "background.jpg";
      …
    }
  },
  mounted() {
    this.ctx =
      this.$refs.cvs.getContext('2d');
    this.drawBoard();
  }
}).mount("#app");
```

Recap: Arrow functions

- New syntax for anonymous functions

```
let add = function(x, y) { return x + y };  // old syntax
```



- Arrow functions use lexical context (= "surrounding code") for *this*
  - suitable whenever dynamic context is not needed
  - should not be used when declaring object methods

Vue Asynchronity Example

```
…
<div id="app">
  <canvas ref="cvs" />
</div>
...
```

```
Vue.createApp({
  data() {return {}},
  methods: {
    drawBoard() {
      let img;
      img = document.createElement("img");
      img.onload = () => {
        this.ctx.drawImage(0,0);
      };
      img.src = "background.jpg";
      …
    }
  },
  mounted() {
    this.ctx =
      this.$refs.cvs.getContext('2d');
    this.drawBoard();
  }
}).mount("#app");
```

Vue Methods revisted:

- Don't use Arrow functions on Vue instance methods
  - this will not work correctly
  - "this"-reference is managed by view, not possible with arrow functions

```
Vue.createApp({
  data: () => {return {}},     // WILL NOT WORK CORRECTLY
  methods: {
    drawBoard: () => {         // DO NOT DO THIS!
      let img;
      img = document.createElement("img");
      img.onload = () => { this.ctx.drawImage(0,0); };  // OK
      img.src = "background.jpg";
      …
    }
  },
  mounted: () => {             // NEVER, EVER!
    this.ctx =
      this.$refs.cvs.getContext('2d');
    this.drawBoard();
  }
}).mount("#app");
```

Web Worker

- Problem: CPU-intensive JavaScript blocks Browser
- Solution: Worker - Threading forJavaScript
  - JavaScript Code of Workers runs in background
  - Communication with main script via messaging

```
// main.js
let worker= new Worker("worker.js");
worker.onmessage= function(event) {
   alert("Worker sent: "+event.data);
};
function callWorker() {
   worker.postMessage("Hello Worker!");
}

//worker.js
this.onmessage = function(event) {
   postMessage("Hello Main Script, you sent: "+event.data);
}
```

Web Worker

- Worker are restricted
  - Same Origin Policy!
  - cannot access dom elements
  - restricted access to javascript APIs

- allowed APIs
  - postMessage / onmessage
  - set event listeners
  - XMLHttpRequest (send Ajax requests)
  - WebSockets (next lecture)
  - Timer
  - can spawn other workers …

- cancel a worker:
  - worker.terminate()

Bad example to stop a worker

- Why is this worker code erroneous?

```
let running= false;
onmessage= function(event) {
  if (event.data=='start') {
    running= true;
    run();
  } else if (event.data=='stop') {
    running= false;
  }
}

function run() {
  while (running){ ... }
}
```